

The important distinction is that the customer is **not uploading a video just to make a 3D world**. They are uploading **their actual autonomy model + their vehicle/robot configuration + sensors + real-world recordings**, and then the system becomes an autonomous training scientist that keeps working without a human.

1. **Customer loads their autonomy model**
  - Driving model / VLA / RL policy / perception + planning stack.
  - Could eventually support cars, robots, etc
2. **They configure the physical system**
  - Vehicle dimensions.
  - Camera positions/FOV.
  - LiDAR if available.
  - Other sensors.
  - Acceleration, braking, steering limits.
3. Tire/vehicle dynamics.
4. **They upload real-world recordings**
  - Camera video.
  - Preferably calibration/pose information.
  - Optional LiDAR/depth/telemetry.
  - Your reconstruction pipeline creates the world.
5. **World Compiler builds the simulation**
  - Gaussian splatting = photorealistic appearance.
  - Geometry/collision representation = physics.
  - Road layout.
  - Cars/pedestrians become controllable actors.
  - Sensors are recreated virtually.
  - Uncertain reconstructed regions get an uncertainty score.
6. **Gemini creates the training plan itself.**

It might start with:

Test normal lane following.

Then:

Test intersection.

Test pedestrian.

Test occluded pedestrian.

Test low-light intersection.

It isn't following a manually written sequence. It chooses what to test based on what the model currently knows.

## 7. The car starts driving autonomously in the reconstructed Gaussian world.

This is where your original idea becomes really interesting.

Suppose the car reaches an intersection.

Pedestrian appears behind a parked van.

**CRASH.**

Instead of merely recording:

Collision detected.

WorldScout investigates it.

Gemini can inspect:

- simulator state,
- model outputs,
- sensor frames,
- trajectory,
- braking,
- perception detections,
- planning decisions,
- collision information,
- video replay.

And conclude something like:

**Failure identified:** late pedestrian detection under partial occlusion.

Perception detected pedestrian 0.72 s too late.

Planning response was otherwise valid.

Confidence: 91%.

Then **no person tells it what to do next.**

That is the important part.

## The autonomous learning loop

WorldScout decides:

Model lacks capability:

**Occluded pedestrian emergence.**

So it automatically creates a curriculum around that weakness.

Instead of blindly replaying the crash 10,000 times, it creates variations:

### **Easy**

Pedestrian visible earlier.



pedestrian slightly more occluded



faster pedestrian



higher ego speed



different parked vehicle



different crossing angle



different lighting



dusk



rain



multiple pedestrians



very difficult hidden pedestrian

The agent keeps evaluating the model.

Maybe:

### **Round 1**

Success: 27%

WorldScout:

Capability remains insufficient. Continue training.

### **Round 5**

Success: 61%

Improvement detected, but failures remain concentrated at >25 mph.

WorldScout creates additional scenarios specifically around 25–35 mph.

### **Round 11**

Training success: 96%.

But importantly, **it should NOT immediately say the problem is solved.**

A separate **Examiner Agent** gets hidden scenarios that the training agent never saw.

Hidden-world score:

**89%.**

Now WorldScout can say:

Occluded-pedestrian capability verified.

Generalization threshold passed.

Capability marked learned.

And then **the car continues driving.**

That is exactly the behavior you're describing.

So if it failed it doesn't fail forever, it's like a tutor which will help you practice and learn until you pass.

Then maybe five minutes later:

Car encounters glare.

Model crosses lane line.

WorldScout detects another weakness.

Newly discovered capability gap:  
lane-boundary estimation under severe sun glare.

And **the entire process starts again automatically.**

So you essentially have:

**DRIVE → FAIL → UNDERSTAND → GENERATE → TRAIN → TEST → PROVE →  
CONTINUE → FAIL SOMEWHERE ELSE → REPEAT**

until the model becomes progressively more capable.

It is like antigravity or cursor for agentic coding but for autonomous and robotic training.  
WorldScout maintains a live map of what the AI has actually demonstrated competence in:

- Lane following ✓
- Normal intersections ✓
- Pedestrian crossing ✓
- Occluded pedestrians ✓
- Night driving ⚠️
- Heavy rain ✗
- Emergency vehicles ✗
- Construction zones ✗
- Unprotected left turn ⚠️

Also the model can set physics and environment in the world, maybe adding wet roads, maybe adding snowy roads.

Now Gemini has a long-term objective:

**Reduce the model's Reality Debt.**

It isn't simply waiting for crashes.

Once it finishes learning one weakness, it can actively ask:

What important capability has this model never demonstrated?

**Then add WorldScout's acquisition feature**

Eventually the system reaches:

Heavy snow driving — **NO TRAINING DATA**

The current reconstructed worlds cannot teach it.

Instead of stopping and asking a human:

Please upload snow videos.

WorldScout itself determines what real-world evidence it needs.

For example:

Required experience:

- multilane urban road
- snow-covered lane markings
- active snowfall
- visibility <100 m
- moving traffic
- forward camera

Then it searches the customer's **licensed recording library**.

If it finds appropriate footage:

**Acquire → reconstruct → validate → simulate → train.**

If it doesn't:

**Reality gap cannot be resolved using current dataset.**

And it can automatically produce a **capture mission**:

Capture 10–20 minutes of snowy multilane driving using cameras A/B/C with these calibration requirements.

That is the part that differentiates you much more from:

**“AI automatically generates harder simulator scenarios.”**

Your agent can effectively decide:

**I don't have enough reality to teach this model. I need a new reality.**

That's a much more interesting concept.

If it keeps failing forever the model will be smart and realize that it will tell real humans something is wrong or this model isn't capable of doing so and not just blindness training, it will help you save computer money as much as possible.

It will help you to find the right server and gpu cheapest and best for the trainings.

And I would **not make the Gaussian reconstruction itself the center of the pitch**.

Gaussian reconstruction is one of your most impressive technologies, but the story should be:

**Give WorldScout an autonomy model and recordings of reality. It builds worlds, puts the model inside them, discovers what the model cannot do, teaches it until it can, independently proves the improvement, discovers what it needs to learn next, and repeats — without a human designing the training curriculum.**

One technical correction I would preserve: **the car visually drives through the Gaussian world, but Gaussian splats should not own the vehicle physics.** Use the Gaussian representation for photorealistic sensor rendering and a deterministic simulation layer for collisions, vehicle dynamics and controllable actors. That will make the technical story much more credible to judges.

# RealityCI

## Self-healing CI/CD for physical AI

**Give RealityCI an autonomy model, a vehicle/sensor configuration, and recordings of reality. It continuously discovers what the model cannot handle, determines why it failed, builds the missing experiences, trains the model, independently verifies the fix, and moves on to the next weakness—with no human designing the curriculum.**

**WorldScout becomes one subsystem inside RealityCI, not the whole project.**

Basic it is just AI that helps you train automatically.

This is the part I'd add beyond what we discussed.

Suppose your autonomous model crashes into a pedestrian.

A weaker system says:

Pedestrian collision.

Train more pedestrian scenarios.

RealityCI should behave like an actual researcher.

It creates hypotheses:

**H1:** pedestrian wasn't detected.

**H2:** pedestrian was detected too late.

**H3:** planner failed to brake.

**H4:** braking controller couldn't execute commanded deceleration.

**H5:** occlusion caused the perception failure.

Then the agent **runs experiments**.

Remove the occluding van.

→ no crash.

Keep van but move pedestrian visibility 1.2 seconds earlier.

→ no crash.

Keep perception output but replace planner with oracle planner.

→ still safe.

Replace perception with oracle perception.

→ safe.

Therefore:

**Causal root cause: perception failure caused by partial occlusion.**

Then it trains *that capability*.

## And then your original Gaussian-world idea becomes important

The customer imports:

**Autonomy Model**

+

**Vehicle**

+

**Sensor Rig**

+

**Real-world recording**

RealityCI compiles that recording into an executable world.

For example:

- Gaussian representation → photorealistic camera rendering
- Metric geometry → collisions
- Physics engine → vehicle behavior



- Actor models → pedestrians/cars
- Calibration → cameras
- Optional actual LiDAR data → LiDAR simulation/calibration
- Road graph → traffic topology
- uncertainty map → determines what portions RealityCI trusts

Then the customer's **actual model drives in it**.

This is where your project looks visually insane in a hackathon demo.

So the main agent's long-term mission becomes:

**Reduce Reality Debt while preventing regressions in previously learned capabilities.**

That's a very understandable objective for judges.

---

## Then WorldScout handles missing reality

Suppose RealityCI determines:

Model cannot drive when snow hides lane markings.

But none of its existing worlds contain snow.

A normal system stops.

RealityCI asks:

**Do I possess the evidence necessary to teach this?**

No.

So **WorldScout Agent** generates an experience requirement:

### **Need**

- multilane roadway
- snow-covered markings
- active snowfall
- moving traffic
- poor contrast
- forward + side cameras
- sufficient camera motion

- calibrated capture

It searches the authorized dataset.

If found:

**Acquire** → **reconstruct** → **validate** → **train**.

If not:

**New reality required.**

And it generates a capture mission.

That is a genuinely interesting agent action.

## Then add one more thing: Regression Memory

Suppose RealityCI fixes pedestrians.

Then trains rain.

But the new model gets worse at pedestrians.

RealityCI should know:

 REGRESSION

Occluded-pedestrian success fell from 94% → 78%.

It **rejects the new model checkpoint** automatically.

Then adjusts training to preserve the earlier capability.

Now you're no longer making an "automatic trainer."

You're building something resembling:

## Autonomous software engineering for learned physical systems.

That is a much bigger idea.

---

## Is this actually attractive to Google / Waymo?

**Strategically, yes. Not because Waymo needs your Gaussian simulator.**

Waymo is already extremely advanced there.

Waymo publicly says its Driver has accumulated billions of simulated miles, and its new World Model is specifically focused on high-fidelity controllable autonomous-driving simulation. Waymo also maintains research around simulation, Sim Agents, scenario generation, end-to-end driving and closed-loop RL.

Waymo has even researched Gemini-based end-to-end driving through **EMMA**.

So a Google/Waymo person could immediately understand the problem.

The interesting question you're answering is not:

How do we simulate a street?

It's:

**Who decides what the autonomy model needs to experience next?**

And:

**Can that entire development cycle itself become autonomous?**

That is more interesting.

## Don't support "models." Support interfaces

This is the important product decision.

# Gemini decides WHICH training to perform

This is where the Google part becomes central.

Suppose a model crashes.

Gemini Scientist determines:

Failure:

pedestrian not detected during occlusion

Model capabilities:

training = LoRA + behavior cloning

Available experiences:

243 relevant scenarios

Available GPU:

1 × A100

Previous attempted training:

LoRA run #7 produced regression

1. Generate 300 occlusion scenarios.
2. Keep 20% as hidden evaluation.
3. Use expert simulator trajectory as labels.
4. LoRA fine-tune action/planning layers.
5. Stop after plateau or regression.
6. Run hidden examiner.

7. Reject checkpoint if old capabilities degrade >3%.

Gemini isn't performing backpropagation.

Gemini is acting as the **research scientist controlling the training system**.

# Final software stack

This is what I would actually build.

| Part                      | Technology               |
|---------------------------|--------------------------|
| Main language             | Python 3.12              |
| ML                        | PyTorch                  |
| Google agents             | Google ADK 2.0 Python    |
| Main reasoning            | Gemini through Vertex AI |
| Coding/self-repair        | Antigravity SDK Python   |
| Desktop UI                | Qt 6 + QML               |
| High-performance renderer | C++ / CUDA               |

|                             |                                      |
|-----------------------------|--------------------------------------|
| Graphics                    | <b>Vulkan</b>                        |
| Gaussian reconstruction     | <b>PyTorch + CUDA/C++</b>            |
| Physics                     | own adapter + CARLA/MuJoCo support   |
| Internal huge sensor frames | <b>shared memory / CUDA memory</b>   |
| Local IPC                   | <b>sockets</b>                       |
| Remote API                  | <b>gRPC + Protobuf</b>               |
| Robot integration           | <b>ROS2</b>                          |
| Model inference             | <b>PyTorch / JAX / ONNX adapters</b> |
| Training                    | <b>Trainer plugins</b>               |
| Agent events                | <b>Google Pub/Sub</b>                |
| Persistent experiment state | <b>Firestore</b>                     |
| Worlds/videos/checkpoints   | <b>Cloud Storage</b>                 |

|                |                                                      |
|----------------|------------------------------------------------------|
| GPU machine    | <b>Compute Engine A100 40GB/80GB or RTX Pro 6000</b> |
| Source control | Git/GitHub                                           |
| Build for C++  | CMake                                                |